The Bucket as Universal Object: A General Theory of Capacity-Constrained Service

Macharia Barii

November 3, 2026

Abstract

The consolidation constant — a departure incurs a fixed cost against a finite capacity, so profitability is governed by load factor — was stated for logistics. This paper argues that the object to which the law applies is not the departure but the **bucket**: a finite capacity, a fixed cost incurred at commitment, a cutoff time, and a kill threshold. The bucket is a **bulk-service queue with an economic threshold**, and the law of consolidation is its profitability condition. This identification connects the logistics theory to seventy years of queueing theory. We state the equivalence formally, survey the known algorithms for bulk-service queues — threshold policies, control-limit policies, dynamic programming, busy-period analysis — and show that each is an algorithmic solution to a problem the logistics theory poses. We then generalise beyond logistics: airlines, cloud computing, hospital scheduling, university enrolment, and restaurant reservations are all bucket-management problems with different parameters. The law of consolidation is not a theory of logistics. It is a theory of **capacity-constrained service with a commitment deadline**, and logistics is the case where the parameters are hardest and the institutions are most visible.

1 From Departure to Bucket

The prior papers in this series analysed the **departure**: a vehicle leaves at a moment in time, carrying N units against a capacity K, at a fixed cost c_0. The law of consolidation states that the departure is profitable if and only if

$$\lambda = \frac{N}{K} > \lambda^* = \frac{c_0}{K(p - c_v)}. \tag{1}$$

This is correct, and it is incomplete. The departure is the *realization* of a decision that was made earlier. The decision is made at a **cutoff time** τ, when the operator must decide whether to commit the fixed cost or kill the departure. Before τ, the operator is managing an **inventory** of committed units against a capacity that is not yet fixed. The object that exists before the departure is the **bucket**.

Definition 1 (Bucket)**.** *A bucket is a quadruple*

$$\mathcal{B} = (K,\ c_0,\ \tau,\ \lambda^*), \tag{2}$$

*where K is capacity, c_0 is the fixed cost incurred at commitment, τ is the cutoff time, and λ^* is the kill threshold. The bucket has a **state** $n(t)$, the number of units committed at time $t < \tau$, with $n(t) \leq K$.*

The bucket-management problem is: given stochastic arrivals, decide which to accept, at what price, and whether to fire or kill at τ. This is a richer object than the departure, and it is the object to which the law of consolidation properly applies.

2 The Queueing Identification

The bucket is not merely analogous to a bulk-service queue. It *is* one.

Definition 2 (Bulk-service queue)**.** *A bulk-service queue is a service system in which arrivals accumulate in a buffer, and a server processes them in batches of fixed or variable size, with a maximum batch size b and a minimum batch size a. The server may initiate service when the queue reaches a threshold Q, and the service incurs a cost c_0 per batch.*

Theorem 1 (The Bucket–Queue Equivalence)**.** *Every bucket $\mathcal{B} = (K, c_0, \tau, \lambda^*)$ is a bulk-service queue with:*

- *maximum batch size $b = K$,*
- *minimum batch size $a = \lceil \lambda^* K \rceil$,*
- *service cost c_0,*
- *threshold policy $Q = a$ (fire when the queue reaches the kill threshold),*
- *and a finite buffer with abandonment at τ.*

Conversely, every bulk-service queue with a service cost and a finite buffer is a bucket.

Proof. The mapping is direct: committed units are queue occupants; capacity is the maximum batch size; the kill threshold is the minimum batch size below which service is not initiated; the fixed cost is the service cost; the cutoff is the buffer abandonment time. The state $n(t)$ is the queue length. The decision to fire or kill is the decision to initiate service or continue idling. □

This identification is the central result of the paper. It means that the logistics theory does not need to develop its own mathematics for bucket management. The mathematics exists, and it is the theory of bulk-service queues.

3 The Law of Consolidation in Queueing Terms

The law of consolidation translates directly.

Law 1 (Consolidation, queueing form)**.** *For a bulk-service queue with batch capacity K, service cost c_0, revenue p per unit, and variable cost c_v per unit, the expected profit per batch of size n is*

$$G(n) = n(p - c_v) - c_0. \tag{3}$$

The batch is profitable if and only if

$$n > n^* = \frac{c_0}{p - c_v}, \tag{4}$$

which is equivalent to $\lambda = n/K > \lambda^* = c_0/[K(p - c_v)]$.

The queueing literature calls n^* the **critical batch size**, and the policy of serving only when the queue reaches n^* is called an **N-policy** or a **control-limit policy**. The logistics literature calls it the **kill rule**. They are the same object.

4 Known Algorithms and Their Logistics Interpretation

The bulk-service queueing literature has developed a family of algorithms for the bucket-management problem. Each has a direct logistics interpretation.

4.1 The N-policy serve and the control limit

The when simplest policy is: the queue reaches N, and continue serving until the queue falls below N. This is the **N-policy**.

Proposition 1 (N-policy optimality)**.** *Under linear holding costs and a fixed service cost, the optimal policy is an N-policy: serve* $\min(n, K)$ *when $n \geq N$, and idle otherwise. The optimal threshold N^* is the smallest integer satisfying*

$$G(N^*) \geq 0 \quad \textit{and} \quad G(N^* - 1) < 0. \tag{5}$$

This is the formal justification for the kill rule. The optimal threshold is exactly the critical batch size, and it is computable in $O(1)$ from the cost parameters. The matatu conductor who waits until the vehicle is "full enough" is implementing an N-policy with an empirically calibrated N^*.

4.2 The (a, c, d) policy and the secondary threshold

In many real systems, once service has started, it is costly to stop and restart. The (a, c, d) policy captures this: start service when the queue reaches c, serve batches of maximum size d, and continue serving even when the queue falls below c, provided it remains above a secondary threshold a.

Proposition 2 ((a, c, d) policy)**.** *For a bulk-service queue with startup cost and a secondary threshold, the optimal policy is of the form (a, c, d): initiate service at c, continue until the queue falls below a, and serve batches of size at most d.*

The logistics interpretation is the **route continuation rule**: once a vehicle has departed, it continues to serve drops until the marginal drop fails the incremental test, even if the route's average load factor has fallen below the threshold. The startup cost (dispatch) is sunk, so the continuation threshold a is lower than the initiation threshold c. This is why the matatu will take a late-arriving passenger at a discount once the vehicle has started moving: the marginal cost of adding them is near zero, and $a < c$.

4.3 Dynamic programming and the SMDP

For stochastic arrivals and multiple decision epochs, the optimal policy is the solution to a **semi-Markov decision process** (SMDP). The state is the queue length; the action is to serve a batch of size b or to wait; the reward is $G(b)$; the transition probabilities are the arrival distribution.

Proposition 3 (SMDP optimality)**.** *The optimal batch size and timing policy is the solution to the Bellman equation*

$$V(n) = \max\left\{ V(n+1) - h,\ \max_{b \leq n} \left[G(b) + V(n-b) \right] \right\}, \tag{6}$$

where h is the holding cost per unit time. The optimal policy is a threshold policy: serve when $n \geq Q^*$, where Q^* is the smallest n for which the serve action dominates the wait action.

The logistics interpretation is the **bucket cutoff decision**: at each review point, the dispatcher compares the value of waiting for one more parcel against the value of firing the bucket now. The optimal cutoff is the threshold Q^*, and it is computable by backward induction for finite horizons or by value iteration for infinite horizons.

4.4 Busy-period analysis and the kill rate

The **busy period** of a bulk-service queue is the time from the start of service until the queue empties. Its distribution determines the fraction of time the server is idle, which in logistics terms is the fraction of buckets that are killed.

Proposition 4 (Busy-period and kill-rate)**.** *The long-run kill rate κ of a bucket system is the fraction of time the queue is below the threshold at the cutoff. It is determined by the busy-period distribution and the arrival rate:*

$$\kappa = \frac{\mathbb{E}[\textit{idle time per cycle}]}{\mathbb{E}[\textit{cycle length}]}. \tag{7}$$

This is the formal version of the kill-rate target from the flywheel paper. The target $\kappa \in [0.10, 0.25]$ is the range in which the busy-period distribution indicates a corridor that is neither over-supplied nor under-supplied. Outside this range, the corridor is either killing too many buckets (demand-starved) or too few (over-supplied).

4.5 The airline yield-management connection

The most important connection is to airline yield management. The airline seat-allocation problem is exactly a bulk-service queue with multiple customer classes and a finite buffer.

Proposition 5 (Yield management as queue control)**.** *The airline yield-management problem — allocating seats among fare classes on a single flight — is equivalent to the optimal control of admission to a bulk-service queue with heterogeneous arrivals. The optimal booking limits are the thresholds of the optimal admission policy.*

This is not an analogy. Subramanian, Stidham, and Lautenbacher formulate the airline problem as a Markov decision process and explicitly note that their approach "exploits the equivalence to a problem in the optimal control of admission to a queueing system, which has been well studied in the queueing-control literature". The logistics bucket is the same object with different parameters: the aircraft is the server, the seats are the batch capacity, the fare classes are the customer types, and the departure time is the cutoff.

5 The Generalization Beyond Logistics

The bucket–queue equivalence means that the law of consolidation is not a theory of logistics. It is a theory of **capacity-constrained service with a commitment deadline**. The following table shows the mapping for several domains.

Domain	K	c_0	τ
Airline	Seats	Fuel + crew + fees	Departure
Freight train	Tonnes	Track access + crew	Departure
Cloud computing	Reserved instances	Provisioning cost	Billing cycle
Hospital	Theatre hours	Surgical team + theatre	Scheduled start
University	Class seats	Instructor + room	Enrolment close
Restaurant	Tables	Kitchen + staff	Reservation time
Locker bank	Doors	Installation + power	Daily pickup

In each case, the same law applies: the batch is profitable above a threshold, the kill rule prevents unprofitable batches, and the optimal threshold is computable from the cost parameters. The institutions differ; the mathematics does not.

6 The Algorithmic Stack

The theory yields a four-layer algorithmic stack, each layer corresponding to a known class of bulk-service queue algorithms.

Layer	Question	Algorithm
Threshold	At what load factor do we fire?	N-policy; critical batch size n^*
Continuation	Once started, when do we stop?	(a, c, d) policy; secondary threshold
Timing	When do we review the queue?	SMDP; control-limit policy Q^*
Admission	Which arrivals do we accept?	Yield management; booking limits

The logistics engine specified in Document 4 implements all four layers:

- The **threshold** is the kill rule λ^*.
- The **continuation** is the marginal insertion test for adding a drop after the route has started.
- The **timing** is the bucket cutoff τ.
- The **admission** is the gate test (address, contribution, risk).

The engine is therefore not a novel algorithm. It is an implementation of known bulk-service queue control policies, applied to the specific parameters of last-mile logistics.

7 What the Generalization Buys

Three things.

First, a mathematical foundation. The logistics theory no longer needs to prove its own theorems. The optimality of the kill rule is a known result in queue control. The busy-period distribution is a solved problem. The yield-management equivalence is established. The logistics contribution is the *parameterisation* — the identification of K, c_0, τ, and λ^* in a specific domain — not the mathematics.

Second, a transfer of algorithms. The queueing literature has efficient algorithms for threshold computation, busy-period analysis, and SMDP solution. These can be applied directly. The logistics engine is a queue controller, and the queue-control literature is its manual.

Third, a scope expansion. The theory is portable. Any domain with finite capacity and a commitment deadline is a bucket domain. The same diagnostic — the utilisation ratio, the kill rate, the threshold — applies. The theory is not about trucks. It is about **commitment under capacity constraints**.

8 Falsifiability

Four observations would refute the bucket–queue equivalence as a useful generalization.

1. **Against the equivalence:** a capacity-constrained service with a commitment deadline whose profitability is not governed by a load-factor threshold.
2. **Against the N-policy:** evidence that a non-threshold policy outperforms the optimal N-policy in a bucket domain.

3. **Against the yield-management equivalence:** evidence that airline seat allocation and logistics bucket management require structurally different mathematics.

4. **Against the portability:** evidence that the utilisation diagnostic fails to predict which constraint binds in a non-logistics bucket domain.

Each is testable in the domain to which it applies.

9 Conclusion

The departure is the realization. The bucket is the object. And the bucket is a bulk-service queue with an economic threshold.

This identification does three things. It gives the logistics theory a mathematical foundation it did not have. It gives the practitioner a library of algorithms — N-policy, (a, c, d) policy, SMDP control limits, busy-period analysis — that solve the bucket-management problem. And it gives the theory a scope that extends far beyond logistics, to any domain where capacity is finite and commitment is temporal.

The law of consolidation is not a law of trucks. It is a law of **batches**. The matatu, the aircraft, the container ship, the locker bank, the operating theatre, the university class, and the cloud instance are all buckets. They obey the same law, and they are managed by the same algorithms.

> **The theory in one sentence:** a bucket is a finite capacity, a fixed cost, a cutoff, and a kill threshold — which is to say, a bulk-service queue with an economic threshold — and the law of consolidation is the condition under which the queue is worth serving.